

Bronze → Silver Pipeline

How the whole system works, end to end — as implemented today

Repo `bronze_to_silver_conversion` · branch `main` · 19 Aug 2026 · written from the code and verified against the live Neon database, with live row counts throughout

1 What the system is

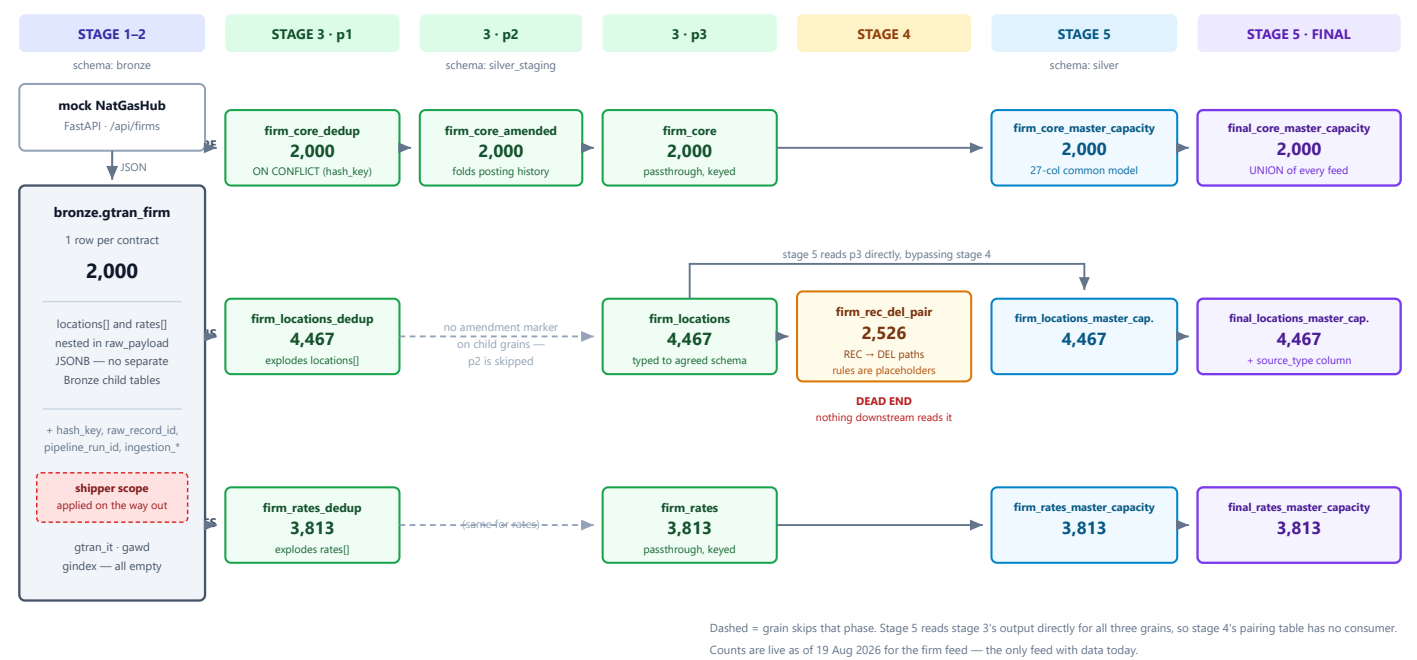
A batch pipeline that pulls natural-gas capacity JSON feeds from NatGasHub, lands them raw in **Bronze**, and refines them through four further stages into consolidated **Silver** master capacity tables. Everything runs on one Postgres database (Neon) and is orchestrated by GitHub Actions. There is no streaming, no external scheduler and no ORM — every transformation is set-based SQL executed server-side.

Two things shape the whole design:

- **One file per output table.** Each table is a Python class that supplies its own `CREATE TABLE` and `INSERT ... SELECT`, and self-registers. There is no central list of tables to maintain, and no orchestration code that knows table names.
- **Feeds are independent all the way through.** Firm, interruptible, awards and IOC each have their own ingest, their own stage runs and their own workflow files. One feed failing or being absent never blocks another; only the three FINAL tables span feeds, and they consolidate whichever feeds happen to exist.

2 End to end

The diagram below is the whole pipeline for one feed (firm), with the live row count in each table. The other feeds have identical shapes — they are simply empty today.



One feed, end to end. Interruptible, awards and IOC have the same shape and no rows.

3 Physical layout

Four schemas on one Neon Postgres database. Schema names are env-driven (`BRONZE_SCHEMA` , `SILVER_SCHEMA` , `DECOMP_SCHEMA`), not hardcoded.

SCHEMA	HOLDS	WRITTEN BY
<code>bronze</code>	Raw landed feeds, one table per feed, plus the run log and the shipper config	Stage 1–2 ingestion
<code>silver_staging</code>	All stage-3 intermediate output (p1 dedup, p2 amendments, p3 decomposition)	Stage 3
<code>silver</code>	Stage-4 pairing output and all stage-5 master capacity tables, per feed and FINAL	Stages 4 and 5
<code>public</code>	Legacy / dashboard tables — <code>workflows</code> , <code>workflow_runs</code> , <code>users</code> , <code>pipeline_exceptions</code> , <code>stage1_validated</code> ... <code>stage5_published</code> , and the reference model <code>final_core_master_capacity</code>	Not by this pipeline

Live table inventory

SCHEMA	TABLE	ROWS	NOTE
<code>bronze</code>	<code>gtran_firm</code>	2,000	Firm contracts
	<code>gtran_it</code>	0	Interruptible — feed exists, never run
	<code>gawd</code>	0	Capacity release awards
	<code>gindex</code>	0	Index of Customers
	<code>ingestion_log</code>	1	One row per ingestion run
	<code>shipper_mapping</code>	0	Shipper scope config (§8)
<code>silver_staging</code>	<code>firm_core_dedup</code>	2,000	p1
	<code>firm_locations_dedup</code>	4,467	p1, exploded
	<code>firm_rates_dedup</code>	3,813	p1, exploded
	<code>firm_core_amended</code>	2,000	p2
	<code>firm_core</code>	2,000	p3
	<code>firm_locations</code>	4,467	p3
	<code>firm_rates</code>	3,813	p3
	<code>awards_core_dedup</code>	0	p1, no source rows

SCHEMA	TABLE	ROWS	NOTE
silver	firm_rec_del_pair	2,526	Stage 4
	firm_core_master_capacity	2,000	Stage 5
	firm_locations_master_capacity	4,467	Stage 5
	firm_rates_master_capacity	3,813	Stage 5
	final_core_master_capacity	2,000	Consolidated — firm only, so far
	final_locations_master_capacity	4,467	Consolidated
	final_rates_master_capacity	3,813	Consolidated
	firm_transport_rate	0	The original worked example. Reads <code>gtran_rates / gtran_loc</code> , which no longer exist — permanently skipped as <i>missing sources</i> . Effectively dead.

4 The stages

Stage 1–2 · Ingestion LIVE

A subproject at `src/transformations/stage_1_2(ingestion)/`, formerly its own repo. A FastAPI **mock NatGasHub** serves JSON fixtures over HTTP; in CI each workflow boots its own copy inside the runner, curls the feed to a file, and hands it to `python -m src.main`. Pointing at the real NatGasHub means replacing the boot+fetch steps with a live call and an API-key secret.

The ingestion itself is five steps: **load** JSON → **validate** → **route** → **transform** → **write**, plus one `ingestion_log` row. Points worth knowing:

- **One Bronze table per feed.** Nested `locations / rates` arrays are *not* fanned out here — they stay on the contract row as JSON text columns and inside `raw_payload` JSONB. Exploding them is stage 3's job.
- **Validation is two-level.** A structural failure aborts the payload; a record missing required fields is still landed but flagged `ingestion_status = 'INVALID'`, so nothing is silently lost.
- **Idempotency is a content hash.** `hash_key` is SHA-256 over the record's canonical business values, metadata excluded. Re-ingesting identical data produces the same hash and the unique constraint makes the load a no-op. This hash is what all of stage 3's deduplication rests on.
- **Adding a feed is one registry entry** in `router.py` — no new code paths.

Stage 3 · Bronze → staging

Four phases, and **order matters** — the folder suffixes carry it, because alphabetical order would run amendments before deduplication. The stage-3 workflows loop the phases explicitly.

PHASE	STATUS	WHAT IT DOES
<code>deduplication(p1)</code> 7 transformations	LIVE	Two mechanisms. Flat: copy the Bronze table, <code>ON CONFLICT (hash_key) DO NOTHING</code> — a row whose content was already seen is dropped, so each run's insert is the delta. Nested: explode <code>raw_payload -> 'locations' / 'rates'</code> with

PHASE	STATUS	WHAT IT DOES
		<code>jsonb_array_elements</code> , one output row per element, hashed on the element's own canonical JSON. This is also the only place Bronze is read — see §8.
<code>ammendments(p2)</code> 2 transformations, core only	LIVE	Folds a contract's posting history into one current row. Classifies each posting NEW / ALL_DATA / CHANGES_ONLY, finds the latest full restatement as <i>baseline</i> , discards everything before it, then overlays later partial postings by taking each column's latest non-null value. Emits provenance (<code>amend_kind</code> , <code>amend_postings_applied</code> , <code>amend_baseline_ts</code>). Only core has amendment markers; locations and rates skip this phase.
<code>decompisition(p3)</code> 6 transformations	LIVE	Produces one stable, keyed table per feed per grain. Locations is typed to the agreed 26-column schema with renames and casts. Core and rates use a generic passthrough that just keys the table — they have no agreed schema yet. Latest-wins dedupe per key is mandatory here, not optional: without it the upsert would touch one target row twice in a statement, which Postgres rejects outright.
<code>standardization(p4)</code>	EMPTY	Folders exist per feed; no code . The workflows already loop over it and it logs a warning and exits 0. Type normalisation that was deferred out of p3 (e.g. parsing <code>loczn</code> into an array) belongs here.

Stage 4 · Rec-del pairing WIRED, RULES PENDING

Turns the flat locations list into one row per receipt→delivery **path**. Unpaired locations are kept and flagged via `pair_status` (UNPAIRED_RECEIPT / UNPAIRED_DELIVERY), so a one-sided contract stays visible rather than vanishing — that is why 4,467 locations produce 2,526 pair rows rather than a smaller matched set.

TWO BUSINESS RULES ARE PLACEHOLDERS

`pair_predicate_sql()` currently returns `TRUE` — **every receipt pairs with every delivery on the same contract**, a cross join within the contract. And `term_columns_sql()` passes the raw term window through, leaving `term_days` and `term_category` NULL. The table shape and the wiring are final; the numbers are not meaningful yet.

STAGE 4'S OUTPUT HAS NO CONSUMER

All three stage-5 locations transformations read `<feed>_locations` — the stage-3 p3 output — **not** `<feed>_rec_del_pair`. So stage 4 currently runs, writes 2,526 rows, exports Parquet, and nothing downstream reads any of it. Either stage 5's locations mapping is meant to consume the paired paths and doesn't yet, or the pairing table is intended as a standalone output. This is worth confirming before the pairing rules are written — the answer changes what they need to produce.

Stage 5 · Master capacity WIRED, MAPPINGS PARTIAL

Two layers. **Per-feed** (12 transformations): map each feed's stage-3 output onto a common column model — this is where `kqtyk` vs `itqtyk` and `firmid` vs `interruptibleid` get normalised to one name. A target column with no mapping is emitted as a typed NULL, deliberately, so every feed's table stays UNION-compatible without inventing values.

FINAL (3 transformations): `UNION ALL` every feed's table per grain into one consolidated table, each row carrying `source_type` so it can be split back apart.

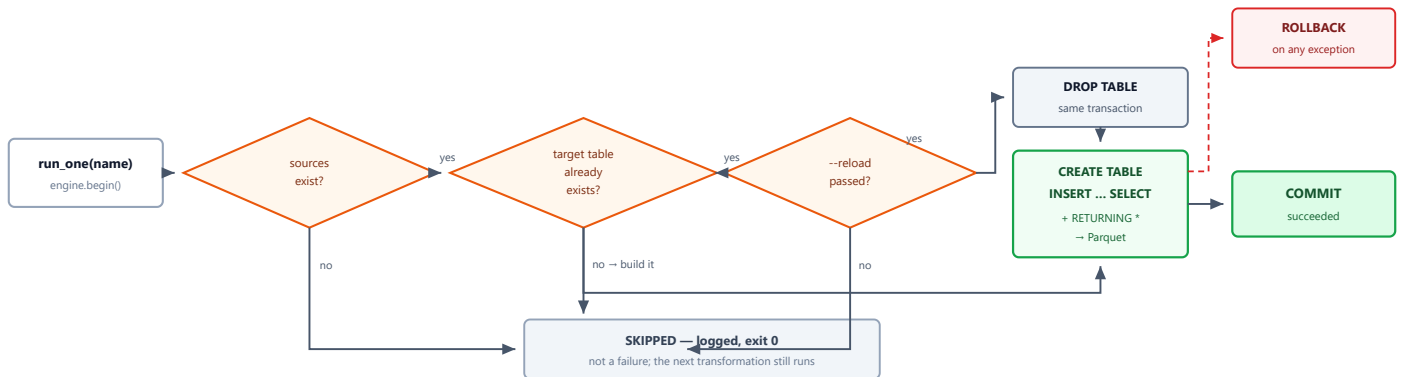
WHY THE FINALS NEVER BLOCK

They do **not** require all feeds. At run time they check which per-feed tables actually exist and union exactly those, naming the absent ones in the log. With no feeds at all they log and return 0 rather than failing. This is what lets a workflow pull any subset of sources — the normal case, not the exception. It is also why the FINAL counts today equal the firm counts exactly.

The column model lives in one file (`models.py`) imported by both layers, because a UNION needs identical column sets and per-subclass copies could silently drift.

5 The transformation framework

Every one of the 31 tables is the same shape. A class sets a name, a target table, and its source tables; implements `create_table_sql()` and `transform_sql()`; and is decorated `@register`. The package auto-imports recursively, so dropping a file in a folder is the entire "adding a table" procedure — there is no list to edit.



What the runner does per transformation. Each one gets its own transaction; a failure isolates rather than stopping the batch.

Four properties that follow from this

- **Per-table transactions.** One transformation failing is logged and the batch continues; the overall exit code is non-zero so a scheduler still flags the job.
- **Load-once, then skip.** If the target table already exists the transformation is skipped entirely — no CREATE, no INSERT. It is only ever populated on the run that creates it. **--reload is therefore not an optimisation flag, it is how the pipeline refreshes at all**, and every stage workflow passes it.
- **Missing sources are a skip, not an error.** This is what lets the awards and interruptible transformations sit registered and dormant, then start working with no code change the moment their Bronze tables have rows.
- **Driver isolation.** Only `db/connection.py` imports SQLAlchemy, so `--list`, `--show-sql` and `--shippers` -style commands work with no database, and swapping backends is one file.

The CLI

```
python run.py --list                # registered transformations (optionally --source FEED)
python run.py --list-groups          # phase folders that contain code
python run.py --show-sql <name>      # print generated SQL, no DB needed
python run.py --inspect              # read-only snapshot: tables + row counts + readiness
python run.py --shippers [--source FEED] # the configured shipper scope
python run.py --table <name> [--reload] # run one
python run.py --group <folder> [--source FEED] [--reload]
python run.py --all [--source FEED] [--reload]
```

`--group` matching is deliberately loose: `stage_4`, `rec_del_pairing`, `stage_4/rec_del_pairing` and `master_capacity/firm/core` all work, so a workflow can address a leaf folder without naming the stage that happens to contain it today.

6 Parquet export

Every table's rows are also written to Parquet from a **single hook** in `base.run()` — the one place any transformation writes — so a new stage or table is exported with no export-code changes.

```
parquet_output/<stage>/<source>/<table>/run_date=YYYY-MM-DD/<run_id>.parquet
```

The export runs **after the INSERT but before COMMIT**: `RETURNING *` is appended to the transform, so the exported rows are the database's own output rather than a reconstruction. Nothing is exported that wasn't durably written, and nothing is written without being exported. In CI each workflow uploads its directory as a build artifact with `if: always()`, so a partial export survives a failed run — which is when it helps most.

CONSEQUENCE WORTH KNOWING

A transformation that skips exports nothing, because it writes nothing. Combined with load-once, that means a run produces no Parquet after the first one unless `--reload` is passed.

7 Orchestration

27 workflow files, all GitHub Actions, all reading the `DATABASE_URL` repo secret and failing fast if it is unset. There are **no cron schedules** — everything is manual dispatch, an orchestrator chain, or a repository dispatch.

GROUP	FILES	PURPOSE
<code>bronze_ingest*.yaml</code>	5	Stage 1–2: one per feed (firm, interruptibles, awards, ioc) plus <code>bronze_ingest.yaml</code> , the fan-out parent that calls all four
<code>(stage3)bronze_to_silver-<feed>.yaml</code>	3	Loops the four p1–p4 phases in order for one feed
<code>(stage4)rec_del_pairing-<feed>.yaml</code>	3	Pairing for one feed
<code>(stage5)master_capacity-<group>_<grain>.yaml</code>	12	4 groups (firm, interruptible, awards, final) × 3 grains
<code><feed>(stage3_4_5).yaml</code>	3	End-to-end orchestrators: ingest → 3 → 4 → 5 → finals, calling the real stage files as reusable workflows
<code>shipper_scope_apply.yaml</code>	1	Replays feeds when the shipper scope changes (§8)

The orchestrators are **compositions, not copies**: each job `uses:` the real per-stage workflow file, so there is exactly one definition of what a stage does and editing it changes every caller. Concurrency groups are scoped per stage *and* feed, so feeds never block each other but a run never collides with itself mid-transaction. Every stage workflow runs `--inspect` before and after, so the run log records inputs and resulting row counts.

ORDERING IS NOT ENFORCED ACROSS STAGES

Outside the three orchestrator chains, nothing stops stage 5 running before stage 4. In particular the FINAL workflows should run *last* — running them before the per-feed stage-5 workflows finish produces finals over incomplete data. It won't error; it will just be wrong. The orchestrators encode the right order and are the safe entry point.

8 Shipper scoping

The dashboard's Shippers panel attaches KHolderName + KHolderNumber (DUNS) pairs to a workflow with an **Add** or **Remove** action, and the pipeline filters the raw feed to them. Rows land in `bronze.shipper_mapping`; the filter is a `WHERE` clause inside the stage-3 p1 `INSERT`.

The reason it lives there and nowhere else: **p1 is the only place in the pipeline that reads a Bronze table**. Everything downstream reads `silver_staging`. So one predicate scopes all seven p1 transformations, every feed and every grain, and every later phase inherits the scope without knowing scoping exists.

MAPPING STATE, PER FEED	EFFECT
No <code>add</code> rows	Unscoped — everything passes. The feature is inert until configured.
One or more <code>add</code> rows	Allow-list — only those DUNS are processed.
Any <code>remove</code> row	That DUNS never passes, even if also added.

Applying a change is a rebuild: `shipper_scope_apply.yml` receives a `shipper-scope-changed` repository dispatch naming the selected feeds and replays those feeds' orchestrator chains, which run with `--reload`.

SCOPE IS PER FEED, NOT PER WORKFLOW

`workflow_id` is stored on the mapping row but the filter never reads it. A shipper attached to firm scopes *every* firm run, whichever workflow triggered it. This is the open design question — a separate document covers the feature in full.

9 What is real and what is scaffolding

31 transformations are registered and the firm feed runs cleanly end to end. But a large share of the *business logic* is still neutral placeholders — **38 SPEC: / TODO markers across 20 files**. The wiring is finished; the rules are not.

AREA	STATUS	GAP
Stage 1–2 ingestion	LIVE	Served by the mock API, not the real NatGasHub.
p1 deduplication	LIVE	Complete.
p2 amendments	LIVE	The NULL-descriptor thresholds were read off a screenshot of the Databricks notebook and should be confirmed against it. May never fire on current data.
p3 decomposition	PARTIAL	Locations is typed to the agreed schema. Core and rates are untyped passthroughs awaiting an agreed schema.
p4 standardization	EMPTY	No code at all. Type normalisation deferred out of p3 has nowhere to live yet.
Stage 4 pairing	WIRED	Both rules are placeholders — pairing is currently a cross join within the contract, and term derivation returns NULL. The 2,526 pair rows are not meaningful business output.
Stage 5 per-feed	PARTIAL	Column maps are incomplete; unmapped targets emit typed NULLs by design, so tables look full while columns are empty.
Stage 5 finals	WIRED	Cross-feed dedupe is a placeholder: the same contract arriving from two feeds keeps both rows, distinguished only by <code>source_type</code> .
IOC / index feeds	NONE	Ingest workflow exists; no stage 3, 4 or 5 transformations are registered . The feed can land in Bronze and go no further.

AREA	STATUS	GAP
Awards / interruptible	DORMANT	Transformations registered, Bronze tables empty. They start working with no code change once ingested. Awards has p1 only — no p2/p3 — so it cannot currently reach stage 4.

THE README IS PARTLY STALE

It describes stage 3 as scaffolded with empty `logic.txt` files and counts 26 workflows and 18 stage-5 files. Stage 3 p1–p3 are now implemented, there are 28 workflows, and 12 stage-5 files (IOC and index have no code). Treat this document's counts as current.

10 Things worth deciding

- Where does master capacity actually target?** `public.final_core_master_capacity` exists as the reference model with PascalCase columns in the `public` schema, while this pipeline writes snake_case into `silver`. Two tables with the same name and different conventions is a trap.
- Is stage 4 supposed to feed stage 5?** Today it doesn't — stage 5 locations reads p3 output directly, leaving the pairing table orphaned. Settle this before writing the pairing rules, since it decides what they must produce.
- The stage-4 pairing rule itself** is a cross join within the contract. Together with the point above, this is the highest-value gap in the pipeline.
- Ordering across stages is by convention.** The orchestrators enforce it; ad-hoc dispatch does not. `workflow_run` triggers would make the dependency real rather than remembered.
- Load-once is a sharp edge.** A table populates only on the run that creates it. Anyone running a stage by hand without `--reload` gets a silent no-op that looks like success.
- Shipper scope granularity** — per feed today, per workflow if the dashboard needs it (§8).
- IOC has no path past Bronze**, and awards has no path past p1. Both need transformations before their ingest workflows produce anything downstream.